

Sesión 2 · Práctica 2.1 · App de solicitudes de vacaciones sobre una tabla de Excel en OneDrive

Lectura y escritura de datos con galerías y formularios, sin SharePoint

Dato	Valor
Duración	60 minutos
Nivel	Inicial-intermedio
Archivos	DATOS/SolicitudesVacaciones_OneDrive.xlsx
Resultado	App de lienzo construida a mano con galería, formulario de alta/edición, eliminación y notificaciones, conectada a una tabla de Excel en OneDrive para la Empresa

Escenario

El departamento de personas gestiona hoy las solicitudes de vacaciones por correo y con una hoja de Excel compartida. No quieren cambiar el sitio donde viven los datos: el Excel se queda, pero deja de circular por correo y pasa a ser el origen de datos de una app. Cada empleado registrará y consultará sus solicitudes desde Power Apps, y la hoja se actualizará sola.

Esta app es la base del **caso integrador del curso**: en la Sesión 3 le añadiremos validación y mejoras de experiencia de usuario, y en la Sesión 4 notificaciones y un flujo de aprobación con Power Automate.

A diferencia de la Práctica 1.1, aquí **no** vamos a dejar que Power Apps genere la app. La construiremos pantalla a pantalla para entender qué hace cada pieza: la conexión, la galería, el formulario y las funciones que leen y escriben datos.

Por qué Excel y no SharePoint. Excel en OneDrive es el origen con el que llega mucha gente a Power Apps: la hoja ya existe y nadie quiere migrarla el primer día. Sirve perfectamente para aprender galerías, formularios y Patch, y tiene límites reales que conviene conocer antes de proponerlo a un cliente. Al final de la práctica los comparamos con lo que ofrece un origen de empresa.

Parte 1 · Preparar el libro en OneDrive

El conector de Excel de Power Apps solo lee **tablas**, no hojas sueltas. Una hoja sin formato de tabla no aparece al conectar.

1. Descarga `DATOS/SolicitudesVacaciones_OneDrive.xlsx`. Ya trae la tabla creada, con el nombre `Solicitudes` y cinco filas de ejemplo.

2. Renómbralo añadiendo tus iniciales, por ejemplo `SolicitudesVacaciones_JL.xlsx`, para que cada alumno trabaje con el suyo.
3. Súbelo a **OneDrive para la Empresa**: entra en `https://www.office.com`, abre **OneDrive** y arrástralo a una carpeta, por ejemplo `Curso Power Apps`.
4. Ábrelo con **Excel para la web** y revisa la estructura:

Columna	Contenido	Tipo en Excel
Id	Identificador único de la solicitud	Texto
Title	Motivo de la solicitud	Texto
FechaInicio	Primer día, en formato <code>aaaa-mm-dd</code>	Texto
FechaFin	Último día, en formato <code>aaaa-mm-dd</code>	Texto
Dias	Días naturales solicitados	Número
Tipo	Vacaciones · Asuntos propios · Formación	Texto
Estado	Pendiente · Aprobada · Rechazada	Texto
Solicitante	Correo del empleado	Texto
Aprobador	Correo del responsable	Texto
Comentarios	Texto libre	Texto

1. Cierra Excel para la web cuando termines de mirarlo. Si dejas el archivo abierto en Excel de escritorio, la app puede no poder escribir.

Para qué sirve la columna Id. Excel no numera las filas como hace una base de datos. La app la rellena al crear la solicitud y, en la Sesión 4, Power Automate la necesita para saber qué fila tiene que actualizar: la acción *Actualizar una fila* pide una columna clave y un valor.

Por qué las fechas van como texto. El conector de Excel no tiene tipos: todo lo que le llega a Power Apps es texto. Una celda con formato de fecha llega como el número de serie de Excel (46236 en lugar del 3 de agosto de 2026), y ordenar o comparar eso es un dolor. Guardándolas como texto `aaaa-mm-dd` se ordenan bien alfabéticamente y se convierten a fecha con `DateValue`. Es el patrón habitual cuando el origen es un Excel, y es la primera diferencia gorda con un origen que sí tiene tipos de columna.

Parte 2 · Crear la app y conectar la tabla

1. En `https://make.powerapps.com`, pulsa **Crear > Aplicación en blanco > Aplicación de lienzo en blanco**. Nombre: `Solicitudes de vacaciones - <iniciales>`. Formato: **Teléfono**.
2. En el menú de creación, pulsa **Datos > Agregar datos**, busca **OneDrive para la Empresa** y selecciona o crea la conexión.

3. Elige tu archivo `SolicitudesVacaciones_<iniciales>.xlsx` y marca la tabla `Solicitudes`. Pulsa **Conectar**.
4. Añade también el conector **Usuarios de Office 365**: lo usaremos para saber quién está conectado.

Qué acabas de conectar. El origen de datos se llama `Solicitudes`, como la tabla, no como el archivo. Power Apps añade por su cuenta una columna oculta, `__PowerAppsId__`, para poder identificar cada fila: es lo que permite editar y borrar registros. No la borres desde Excel.

Otra vía. Desde la propia hoja, *Insertar > Aplicaciones para Office* o el generador automático de Power Apps crean una app de tres pantallas en segundos. Es útil para prototipos y lo verás en la demo del formador, pero hoy construimos la app a mano para entender cada pieza.

Parte 3 · Pantalla de lista con galería

1. Renombra `Screen1` como `scrList`.
2. Añade una cabecera como en la Práctica 1.2 con el texto `"Mis solicitudes"`.
3. Inserta una **Galería vertical** y, al preguntarte por el origen de datos, elige `Solicitudes`. Renómbrala `galSolicitudes`.
4. En el panel de propiedades cambia el **Diseño** a *Título, subtítulo y cuerpo*.
5. Configura los textos de la plantilla (selecciona el primer elemento de la galería para editar la plantilla):

Control	Propiedad Text
Título	<code>ThisItem.Title</code>
Subtítulo	<code>Text(DateValue(ThisItem.FechaInicio); "dd/mm/yyyy") & " – " & Text(DateValue(ThisItem.FechaFin); "dd/mm/yyyy")</code>
Cuerpo	<code>ThisItem.Tipo & " · " & ThisItem.Estado</code>

1. Fíjate en las dos diferencias con un origen con tipos: las fechas hay que convertirlas con `DateValue` antes de darles formato, y `Tipo` y `Estado` son texto plano, así que **no** llevan `.Value`.
2. Cambia la propiedad **Items** de la galería para que cada empleado vea solo sus solicitudes, de la más reciente a la más antigua:

```
SortByColumns(
    Filter(Solicitudes; Solicitante = User().Email);
    "FechaInicio"; SortOrder.Descending
)
```

1. Verás un triángulo amarillo con un aviso de **delegación**. Léelo y no lo ignores: aquí siempre aparece.

Delegación con Excel: no hay. El conector de Excel **no delega nada**. Power Apps trae los primeros 500 registros (ampliable a 2.000 en *Configuración > General*) y hace el filtro y la ordenación en el dispositivo. Con veinte solicitudes de clase no lo notarás; con 5.000 filas, la app mostraría datos incompletos sin avisar al usuario. Esta es la segunda diferencia gorda: un origen de empresa delega el filtrado al servidor y no tiene ese techo. Lo trabajaremos en la demo.

Parte 4 · Pantalla de formulario para crear y editar

1. Pulsa **Nueva pantalla > En blanco** y llámala `scrForm`. Copia la cabecera y cambia el texto por `"Solicitud"`.
2. Inserta un **Formulario de edición** y renómbralo `frmSolicitud`.
3. Configura sus propiedades:

Propiedad	Valor
DataSource	<code>Solicitudes</code>
Item	<code>galSolicitudes.Selected</code>
DefaultMode	<code>FormMode.Edit</code>

1. Pulsa **Editar campos** y añade `Title`, `Tipo`, `FechaInicio`, `FechaFin`, `Dias` y `Comentarios`. No añadas `Estado`, `Solicitante` ni `Aprobador`: esos los rellenará la app o el flujo, no el empleado.
2. Como el origen es texto, Power Apps pinta todas las tarjetas como cuadros de texto. Vamos a arreglar las tres que lo necesitan.
3. **Tarjeta Tipo.** Desbloquéala desde *Avanzado*, borra el cuadro de texto que trae e inserta un **Desplegable** dentro de la tarjeta. Configúralo:

Propiedad	Valor
Items (del desplegable)	<code>["Vacaciones"; "Asuntos propios"; "Formación"]</code>
Default (del desplegable)	<code>Parent.Default</code>
Update (de la tarjeta)	<code>Dropdown1.Selected.Value</code>

1. **Tarjetas FechaInicio y FechaFin.** Desbloquéalas, borra el cuadro de texto e inserta un **Selector de fecha** en cada una. En cada tarjeta:

Propiedad	Valor
DefaultDate (del selector)	<code>If(IsBlank(Parent.Default); Today(); DateValue(Parent.Default))</code>
Update (de la tarjeta)	<code>Text(DatePicker1.SelectedDate; "yyyy-mm-dd")</code>

1. Los nombres `Dropdown1` y `DatePicker1` pueden ser otros en tu app; usa los tuyos. Renómbralos a `drpTipo`, `dtInicio` y `dtFin` para no perderte.
2. **Tarjeta Dias.** Desbloquéala y pon en la propiedad **Default** del cuadro de texto que contiene:

```
DateDiff(dtInicio.SelectedDate; dtFin.SelectedDate; TimeUnit.Days) + 1
```

1. Para que el solicitante se guarde aunque no esté en el formulario, añade la tarjeta **Solicitante**, desbloquéala, oculta la tarjeta (`Visible = false`) y pon en su propiedad **Update**:

```
User().Email
```

Compara esta línea con la del sprint anterior del temario. En un origen con columnas de persona, guardar al solicitante exige construir un registro con `@odata.type`, `Claims`, `DisplayName` y `Email`. Aquí es una cadena de texto. Ganas en sencillez y pierdes todo lo demás: el origen no sabe que eso es una persona, así que nadie valida que el correo exista, no hay foto, no hay búsqueda del directorio y no puedes filtrar por el responsable de alguien. En la Sesión 3, cuando integremos el directorio de Microsoft 365, esta diferencia se va a notar.

Parte 5 · Conectar las dos pantallas

1. En `scrList`, inserta un icono **Agregar** (+) en la cabecera. En su **OnSelect**:

```
NewForm(frmSolicitud);; Navigate(scrForm; ScreenTransition.Cover)
```

1. Selecciona la galería y en su **OnSelect**:

```
EditForm(frmSolicitud);; Navigate(scrForm; ScreenTransition.Cover)
```

1. En `scrForm`, inserta un icono **Marca de verificación** en la cabecera con **OnSelect** `SubmitForm(frmSolicitud)`.
2. Inserta un icono **Cancelar** (X) con **OnSelect** `ResetForm(frmSolicitud);; Back()`.
3. Como `Estado` e `Id` no están en el formulario y Excel no tiene valores predeterminados, hay que ponerlos desde la app. En **OnSuccess** del formulario, si la solicitud es nueva, márcala como pendiente y dale un identificador:

Propiedad de frmSolicitud	Valor
OnSuccess	<code>If(IsBlank(Self.LastSubmit.Id); Patch(Solicitudes; Self.LastSubmit; { Id: "SOL-" & Text(Now(); "yyyymmddhhmmss"); Estado: "Pendiente" }));; Notify("Solicitud guardada: " & Self.LastSubmit.Title; NotificationType.Success);; Back()</code>
OnFailure	<code>Notify("No se ha podido guardar. Revisa los campos y que el archivo no esté abierto en Excel."; NotificationType.Error)</code>

1. Inserta un icono **Papelera** en la cabecera de `scrForm`, visible solo al editar (`Visible = frmSolicitud.Mode = FormMode.Edit`) y con **OnSelect**:

```
Remove(Solicitudes; galSolicitudes.Selected);; Notify("Solicitud eliminada"; NotificationType.Warning);; Back()
```

Tercera diferencia gorda: no hay valores predeterminados, ni columnas obligatorias, ni identificadores. Un origen de empresa puede exigir que una columna tenga valor y rellenar *Pendiente* por defecto. Un Excel acepta cualquier cosa, incluida una fila vacía. Tampoco hay un ID autonumérico. Todo lo que quieras garantizar tienes que garantizarlo en la app, como acabamos de hacer con `Estado` y con `Id`. Guárdalo para la Práctica 3.1, que va justo de eso.

Parte 6 · Escribir sin formulario con Patch

Los formularios son la forma más rápida de escribir datos, pero a veces necesitas modificar un registro con un solo botón. Para eso existe `Patch`.

1. En la plantilla de la galería de `scrList`, inserta un icono pequeño de **Cancelar** a la derecha de cada solicitud.
2. Haz que solo se vea si la solicitud está pendiente: **Visible** = `ThisItem.Estado = "Pendiente"`.
3. En su **OnSelect**, cambia la solicitud a rechazada con un comentario:

```
Patch(Solicitudes; ThisItem; { Estado: "Rechazada"; Comentarios: "Cancelada por el solicitante" })
```

1. Abre el Excel en OneDrive y comprueba que el cambio se ha guardado en la fila correspondiente.

Parte 7 · Probar

1. Ejecuta la app en vista previa y crea una solicitud nueva del 1 al 5 del mes siguiente. Comprueba que `Dias` muestra 5.
2. Edita la solicitud y cambia el motivo.

3. Cancela una solicitud pendiente desde la galería.
4. Abre el archivo en Excel para la web y verifica que el solicitante es tu correo, que la solicitud nueva tiene `Id` y estado *Pendiente*, y que las fechas se han guardado como `aaaa-mm-dd`.
5. Guarda y publica la app.

Comprobación

- La galería muestra solo las solicitudes del usuario conectado, ordenadas por fecha, con las fechas en formato `dd/mm/yyyy`.
- Se pueden crear, editar y eliminar solicitudes desde la app, y los cambios aparecen en el Excel de OneDrive.
- `Días` se calcula solo, el solicitante se guarda automáticamente y una solicitud nueva queda con `Id` propio y estado *Pendiente*.
- Tras guardar aparece una notificación y la app vuelve a la lista.
- El botón de cancelar usa `Patch` sin pasar por el formulario.
- Sabes explicar por qué aparece el aviso de delegación y qué implica.

Qué cambia si mañana el origen es otro

La app que acabas de construir tiene la lógica separada del origen: galería, formulario y `Patch` funcionan igual contra cualquier tabla. Esto es lo que cambiaría al mover los datos a un origen de empresa, y es la conversación que tendrás con cualquier cliente que empiece con un Excel.

Aspecto	Excel en OneDrive	Origen de empresa (lista de SharePoint, Dataverse)
Tipos de columna	Todo texto: <code>DateValue</code> , <code>Value</code> , <code>Text(...; "yyyy-mm-dd")</code>	Fecha, número, elección y persona reales
Elecciones	Desplegable con una lista escrita en la app	Columna de elección; en la fórmula, <code>.Value</code>
Personas	Un correo en texto	Columna de persona con nombre, foto y correo
Valores predeterminados y obligatorios	No existen: los pone la app	Los define el origen
Delegación	Ninguna: techo de 500 o 2.000 filas	Filtra el servidor, sin techo práctico
Uso simultáneo	El archivo se bloquea si alguien lo tiene abierto en Excel	Varios usuarios a la vez sin problema
Permisos	Los del archivo en OneDrive, todo o nada	Por lista, por elemento o por rol de seguridad
Identificadores	Una columna <code>Id</code> que rellena la app	ID autonumérico del propio origen
Automatización	Sin desencadenador de fila creada: flujo desde la app o programado	Flujo automático al crear o modificar un elemento

El resto del curso sigue sobre este Excel. Las prácticas 3.2, 4.1 y 4.2 continúan con este mismo archivo: el aprobador se guardará como correo, las notificaciones saldrán de un flujo lanzado desde la app y la aprobación, de un flujo programado. Guarda bien la dirección de tu archivo en OneDrive.

Retos opcionales

1. Añade un cuadro de búsqueda sobre la galería que filtre por motivo con `StartsWith(Title; txtBuscar.Text)` y comprueba que el aviso de delegación sigue apareciendo, a diferencia de lo que pasaría en SharePoint.
2. Colorea el cuerpo de cada elemento de la galería según el estado: verde si está aprobada, rojo si está rechazada y ámbar si está pendiente, con `Switch(ThisItem.Estado; "Aprobada"; Green; "Rechazada"; Red; RGBA(230; 150; 0; 1))`.
3. Muestra en la cabecera el total de días solicitados este año con `Sum(Filter(galSolicitudes.AllItems; Year(DateValue(FechaInicio)) = Year(Today())); Value(Dias))`.
4. Impide guardar una solicitud cuya fecha de fin sea anterior a la de inicio: pon el icono de guardar en `DisplayMode.Disabled` cuando `dtFin.SelectedDate < dtInicio.SelectedDate`.